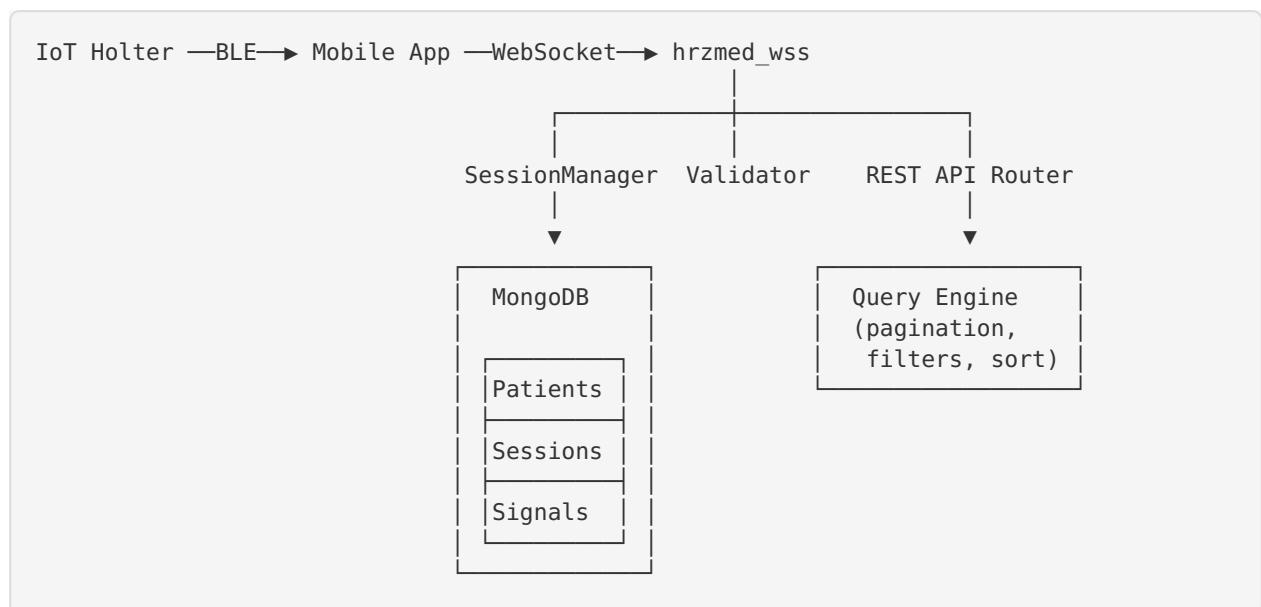


ECG Data Persistence — Fase 6

Overview

Phase 6 implements complete ECG data persistence in MongoDB for the Horizon Medical platform. The system captures real-time ECG signals from IoT Holter devices via WebSocket, stores them in structured collections with optional compression, and provides REST APIs for historical data retrieval.

Architecture



Database Schema

Collection: `patients`

Stores patient demographics and device associations.

Field	Type	Required	Description
patientId	String		Unique external identifier (indexed)
firstName	String		First name (max 100 chars)
lastName	String		Last name (max 100 chars)
dateOfBirth	Date		Date of birth
gender	Enum		male, female, other, unknown
medicalRecordNumber	String		MRN (indexed)
attendingPhysician	String		Physician name
diagnosis	String		Diagnosis text
notes	String		Clinical notes (max 2000)
assignedDevices	Array		Device assignment history
active	Boolean		Active flag (default: true)
createdAt	Date	auto	Mongoose timestamp
updatedAt	Date	auto	Mongoose timestamp

Indexes: patientId (unique), medicalRecordNumber , lastName+firstName , createdAt

Collection: `ecg_sessions`

Represents a continuous recording session from a single device.

Field	Type	Required	Description
sessionId	String	✓	UUID (unique, indexed)
patientId	String		References Patient.patientId
connectionId	String		WebSocket connection that created it
device.deviceId	String	✓	Source device identifier
device.firmwareVersion	String		Firmware version string
startedAt	Date	✓	Session start time (indexed)
endedAt	Date		Session end time
durationMs	Number		Auto-computed duration in ms
status	Enum		recording, completed, interrupted, error
quality.totalSamples	Number		Total ADC samples received
quality.droppedPackets	Number		Dropped packet count
quality.channelsRecorded	[String]		Channel UUIDs that sent data
quality.compressionRatio	Number		Storage compression ratio
signalCount	Number		Number of ECGSignal chunks
tags	[String]		User-defined tags
notes	String		Session notes

Indexes: `sessionId` (unique), `patientId+startedAt` , `device.deviceId+startedAt` , `status+startedAt`

Collection: `ecg_signals`

Individual signal data chunks (one per channel per packet batch).

Field	Type	Required	Description
sessionId	String	✓	References ECGSession.sessionId
channelId	Enum	✓	BLE UUID: 8171-8178
channelIndex	Number		0-based channel index
timestamp	Date	✓	Sample batch timestamp
sequenceNumber	Number		Packet sequence for ordering
samples	[Number]		Raw ADC samples (when uncompressed)
sampleCount	Number	✓	Number of samples (always set)
compressed	Boolean		Whether data is compressed
compressedData	Buffer		Binary compressed data
compression-Meta.algorithm	String		Compression algorithm (delta-rle)
compression-Meta.originalBytes	Number		Original size in bytes
compression-Meta.compressedBytes	Number		Compressed size in bytes
compression-Meta.firstSample	Number		Anchor value for delta decoding
deviceId	String	✓	Source device identifier

Indexes: sessionId+channelId+timestamp , sessionId+sequenceNumber , deviceId+timestamp

Signal Compression

Algorithm: Delta + Run-Length Encoding (delta-rle)

ECG signals from the ADS1298 AFE are 24-bit signed integers. The compression pipeline:

1. **Delta Encoding:** Store differences between consecutive samples. ECG signals are quasi-periodic, so deltas cluster near zero.
2. **Run-Length Encoding:** Collapse runs of identical deltas into (value, count) pairs.
3. **Binary Serialization:** Pack RLE pairs into a compact Buffer (int32 + uint16 per pair).

```
Raw:    [1000, 1005, 1003, 998, 1002]
Deltas: [5, -2, -5, 4]      ↩ small values
RLE:    [(5,1), (-2,1), (-5,1), (4,1)] ↩ no runs in this example
Binary: [4 header bytes] + [6 bytes ↩ 4 pairs] = 28 bytes
vs Raw: 5 ↩ 4 bytes = 20 bytes (no gain for unique deltas)

Constant signal [500, 500, 500, ..., 500] (1000 samples):
Deltas: [0, 0, 0, ..., 0] (999 zeros)
RLE:    [(0, 999)] ↩ 1 pair
Binary: 4 + 6 = 10 bytes vs 4000 bytes = 400↩ compression
```

Typical ECG compression ratio: 1.5× to 4× depending on signal variability.

Configuration

Environment Variable	Default	Description
ECG_COMPRESSION_ENABLED	true	Enable/disable compression
ECG_FLUSH_BUFFER_SIZE	50	Chunks buffered before DB write
ECG_FLUSH_INTERVAL_MS	10000	Periodic flush interval (ms)
ECG_SESSION_INACTIVITY_MS	60000	Inactivity timeout to close session

REST API Endpoints

All endpoints are prefixed with /api/v1 and return JSON.

Sessions

GET /api/v1/sessions

List ECG sessions with filtering and pagination.

Query Parameters:

Param	Type	Description
page	int	Page number (default: 1)
limit	int	Items per page (default: 20, max: 100)
sort	string	Sort fields: startedAt , -startedAt , etc
patientId	string	Filter by patient ID
deviceId	string	Filter by device ID
status	string	Filter by status (recording/ completed/etc)
startDate	string	Filter: sessions started after (ISO date)
endDate	string	Filter: sessions started before (ISO date)
minDuration	int	Minimum duration in seconds
maxDuration	int	Maximum duration in seconds

Response:

```
{
  "data": [{ "sessionId": "...", "device": {...}, ... }],
  "pagination": { "page": 1, "limit": 20, "total": 42, "totalPages": 3 }
}
```

GET /api/v1/sessions/:sessionId

Get a single session by ID.

GET /api/v1/sessions/:sessionId/signals

Get signal chunks for a session.

Query Parameters:

Param	Type	Description
page	int	Page number
limit	int	Items per page
channelId	string	Filter by channel (8171-8178)
decompress	string	Set to false to skip decompression

Response: Includes decompressed samples arrays by default.

GET /api/v1/sessions/active

Get currently recording sessions (in-memory state).

Patients

GET /api/v1/patients

List patients with search and pagination.

Query Parameters:

Param	Type	Description
page	int	Page number
limit	int	Items per page
search	string	Search by name, patientId, or MRN
active	string	Filter by active status (true/false)

GET /api/v1/patients/:patientId

Get patient details including session summary.

POST /api/v1/patients

Create or update a patient (upsert by patientId).

Request Body:


```
{
  "patientId": "PAT-001",
  "firstName": "Juan",
  "lastName": "García",
  "dateOfBirth": "1985-03-15",
  "gender": "male",
  "medicalRecordNumber": "MRN-12345",
  "attendingPhysician": "Dr. López",
  "diagnosis": "Arritmia sinusal",
  "notes": "Seguimiento mensual"
}
```

Statistics

GET /api/v1/stats

Aggregated platform statistics.

Response:

```
{
  "data": {
    "totalSessions": 150,
    "activeSessions": 3,
    "totalPatients": 42,
    "totalSignalChunks": 125000,
    "averageDurationMs": 1800000,
    "totalRecordingTimeMs": 270000000
  }
}
```

Session Lifecycle

Device connects via WebSocket



First ecg_data message arrives



SessionManager.recordECGData()

- ☐ Creates ECGSession (status: "recording")
- ☐ Starts inactivity timer (60s default)
- ☐ Starts periodic flush timer (10s default)



Subsequent ecg_data messages

- ☐ Buffer **signal** chunks **in** memory
- ☐ Reset inactivity timer
- ☐ Flush buffer when full (50 chunks default)



Session ends when:

- ☐ WebSocket disconnects ☐ closeAllSessions()
- ☐ Inactivity timeout ☐ auto-finalize
- ☐ Server shutdown ☐ destroy()



Finalize: flush remaining buffer ☐ update status/duration ☐ remove from memory

File Structure

```

hrzmed_wss/
├── src/
│   ├── models/
│   │   ├── index.js          # Model exports
│   │   ├── Patient.js       # Patient schema
│   │   ├── ECGSession.js    # Session schema
│   │   └── ECGSignal.js     # Signal chunk schema
│   ├── services/
│   │   ├── sessionManager.js # Real-time session management
│   │   └── signalCompressor.js # Delta+RLE compression
│   ├── routes/
│   │   └── ecgRoutes.js      # REST API handler
│   ├── middleware/
│   │   ├── auth.js
│   │   └── rateLimiter.js
│   ├── validators/
│   │   └── ecgValidator.js
│   ├── utils/
│   │   └── logger.js
│   └── tests/
│       ├── signalCompressor.test.js # 23 compression tests
│       ├── persistence.test.js      # 15 DB persistence tests
│       ├── auth.test.js
│       └── ecgValidator.test.js
├── docs/
│   ├── ECG_PERSISTENCE.md        # This document
│   └── index.js                  # Main server (updated)

```

Testing

```

# Run all tests
npm test

# Run specific test suites
node --test tests/signalCompressor.test.js # 23 tests
node --test tests/persistence.test.js      # 15 tests (requires mongodb-memory-server)

```

Test coverage:

- **Signal compression:** Delta encoding, RLE, binary serialization, roundtrip integrity
- **Schema validation:** Required fields, enums, unique constraints, computed fields
- **DB operations:** CRUD, compression through store/retrieve cycle, date filtering, pagination
- **Query patterns:** Range queries, compound filters, sorting

Environment Variables (New)

Variable	Default	Description
ECG_COMPRESSION_ENABLED	true	Enable delta+RLE signal compression
ECG_FLUSH_BUFFER_SIZE	50	Signal chunks before flush to MongoDB
ECG_FLUSH_INTERVAL_MS	10000	Periodic flush timer (ms)
ECG_SESSION_INACTIVITY_MS	60000	Auto-close session after inactivity (ms)
CORS_ORIGIN	*	CORS origin for REST API